Name-Vineet Gajwani

Roll no-2401028, Batch-T12

Subject-ASD&D

Experiment 9

Provisioning AWS Infrastructure using Terraform (Infrastructure as Code)

Lab Outcomes

LO4 : Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

LO5 : Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

LO6 : Develop and demonstrate an end-to-end DevOps pipeline mini-project integrating all stages of DevOps from code commit to deployment and monitoring.

Aim

To install and configure Terraform on Windows, authenticate it with an AWS account, write Terraform configuration files, and provision (and finally destroy) an Amazon EC2 instance using the Terraform workflow of init, validate, plan, apply and destroy.

Theory

Infrastructure as Code (IaC)

Infrastructure as Code is the practice of describing servers, networks, storage and other cloud resources in text files instead of creating them by hand in a web console. Because the infrastructure is defined in code, it can be version-controlled, reviewed, reused and recreated on demand, which makes deployments repeatable and removes configuration drift between environments.

What is Terraform?

Terraform, developed by HashiCorp, is an open-source IaC tool. Infrastructure is written in a declarative language called HCL (HashiCorp Configuration Language): the user states the desired end state, and Terraform works out the API calls needed to reach it. Through providers it supports AWS, Azure, Google Cloud, Kubernetes and many other platforms, so one workflow serves every environment.

Key Terraform Concepts

Concept	Description
Provider	Plugin that lets Terraform talk to a platform's API. In this experiment: hashicorp/aws.
Resource	A single infrastructure object to create, e.g. aws_instance for an EC2 virtual machine.
Output	Values printed after apply (instance ID, public IP, …) defined in outputs.tf.
State file	terraform.tfstate records what Terraform has created; it is compared with the code on every plan.
Lock file	.terraform.lock.hcl pins the exact provider versions so every run uses the same plugins.
Configuration	All .tf files in one folder are read together as a single configuration; file names do not decide what runs.

Terraform Workflow

Command	Purpose
terraform init	Initialises the working directory and downloads the required provider plugins.
terraform validate	Checks the configuration for syntax errors and internal consistency.
terraform plan	Shows the execution plan – what will be created, changed or destroyed – without making changes.
terraform apply	Executes the plan and creates the real infrastructure after confirmation (yes).
terraform output	Displays the output values stored in the state file.
terraform destroy	Removes every resource managed by the configuration so that no cost keeps accruing.

Tools and Requirements

- AWS account with permission to create EC2 instances and the related networking resources
- Windows machine with Command Prompt / PowerShell
- Terraform (Windows AMD64 package) and the AWS CLI
- An IAM user with programmatic access (access key) for Terraform to authenticate with

Procedure and Observations

Step 1: Create or verify your AWS account

1. Open the AWS Management Console.
2. Sign in to your AWS account.
3. Select the Mumbai region (ap-south-1).
4. Open the EC2 Dashboard and check that your account is ready to launch instances.

The account must have permissions to create EC2 instances and use the required networking resources.

Step 2: Install Terraform on Windows

1. Visit the official Terraform installation page.
2. Download the Windows AMD64 ZIP package.
3. Extract terraform.exe into a folder, for example C:\Terraform.
4. Add C:\Terraform to the Windows PATH environment variable.
5. Open a new Command Prompt or PowerShell window and run terraform -version.

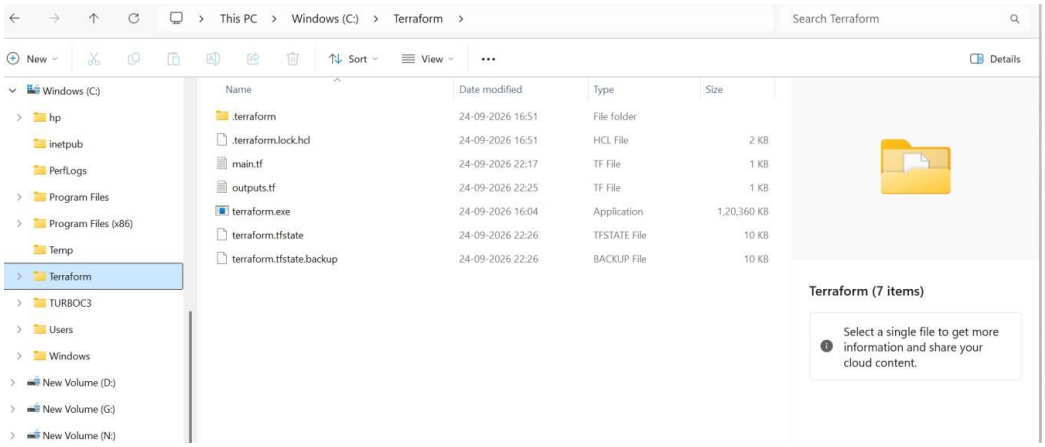


Figure 1: terraform.exe extracted into C:\Terraform (the folder later also holds main.tf, outputs.tf and the state files).

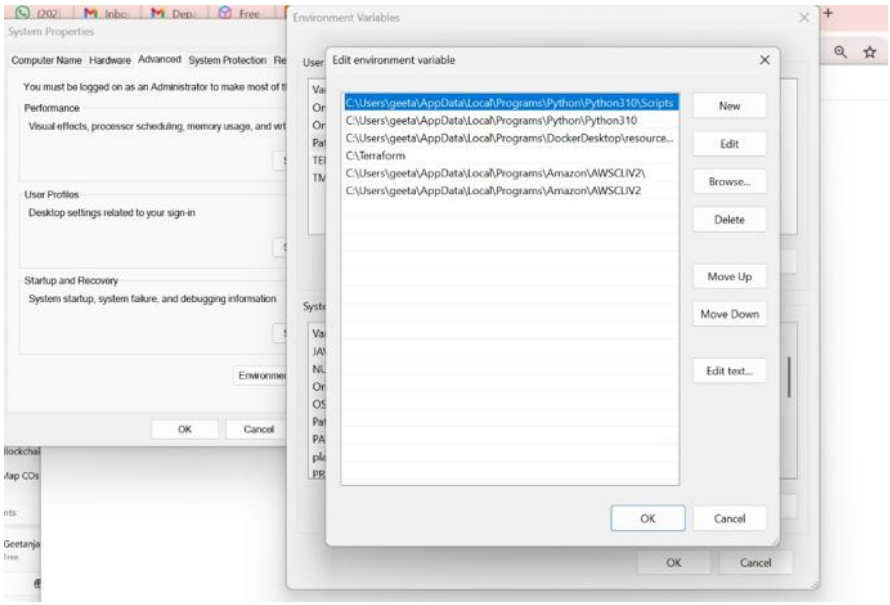


Figure 2: C:\Terraform added to the PATH environment variable so Terraform runs from any directory.

```
PS C:\Users\Vineet Gajwani> terraform -version
Terraform v1.16.4
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.16.5. You can update by downloading from https://developer.
hashicorp.com/terraform/install
PS C:\Users\Vineet Gajwani>
```

Figure 3: terraform -version confirming Terraform v1.16.4 on windows_amd64.

Step 3: Create an IAM user and configure AWS credentials

1. In the IAM console, create a user for Terraform (terraform-user) and attach the permissions policies it needs (for example AmazonEC2FullAccess and AmazonVPCFullAccess).
2. Open the user, choose Create access key and select the Command Line Interface (CLI) use case.
3. Save the access key ID and secret access key securely – the secret is shown only once.
4. Run aws configure and enter the keys, the region (ap-south-1) and the output format (json).
5. Verify the login with aws sts get-caller-identity.

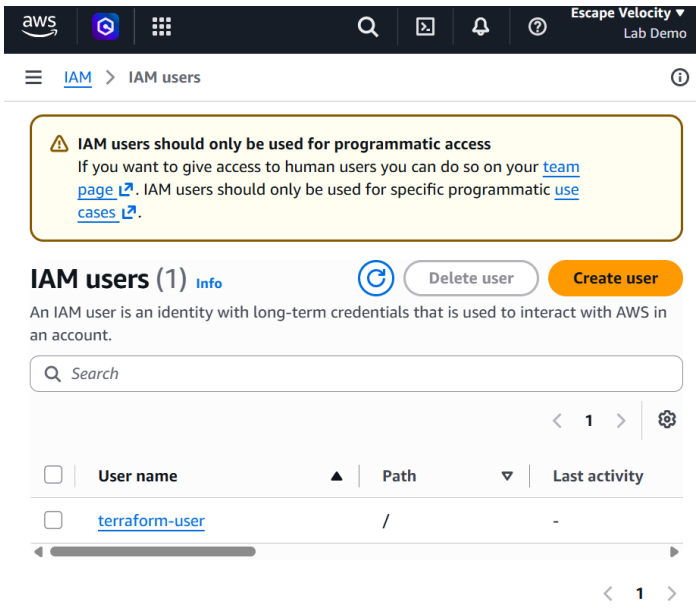


Figure 4: IAM users list showing the terraform-user identity created for Terraform.

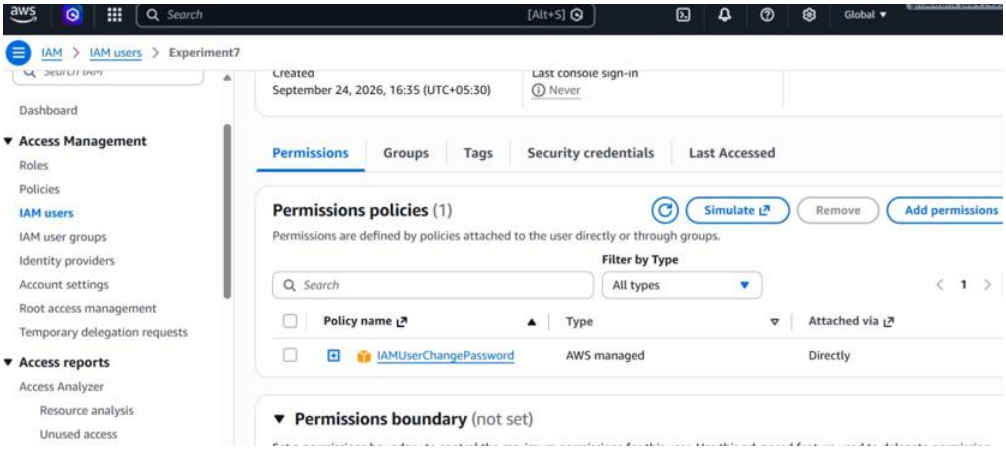


Figure 5: Permissions policies attached to the IAM user.

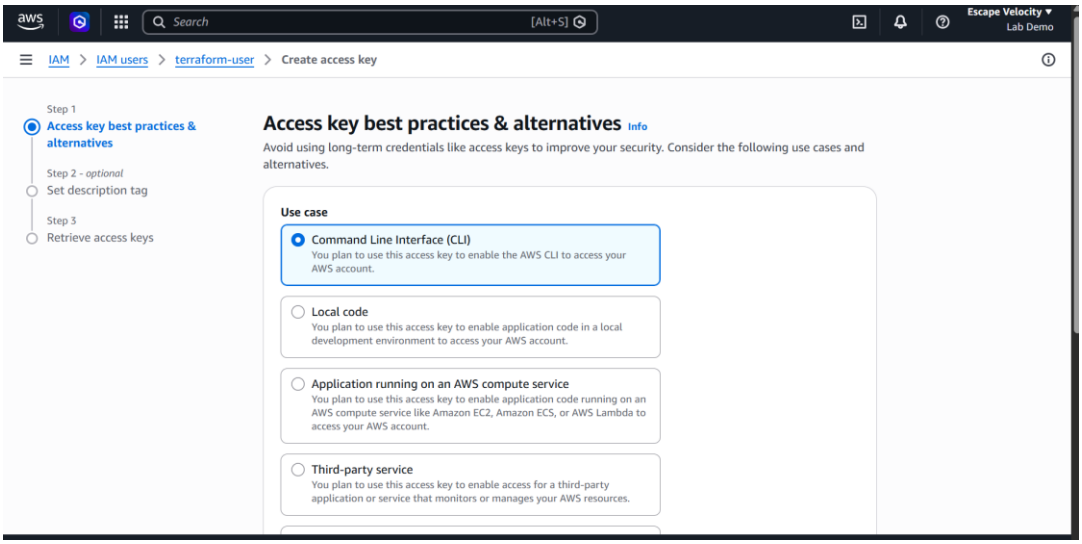


Figure 6: Create access key – Command Line Interface (CLI) selected as the use case.

```
PS C:\Users\Vineet Gajwani> aws sts get-caller-identity
{
    "UserId": "AIDA3MMY7IWMNXT4R5ERJ",
    "Account": "782541735320",
    "Arn": "arn:aws:iam::782541735320:user/terraform-user"
}

PS C:\Users\Vineet Gajwani>
```

Figure 7: aws sts get-caller-identity confirms the CLI is authenticated as user/terraform-user.

Security note: Access keys are long-term credentials. They must never be committed to a repository or shared; delete or deactivate them once the experiment is complete.

Step 4: Write the Terraform configuration files

Two files are created in the working folder. main.tf declares the provider and the EC2 resource; outputs.tf declares the values to be printed after apply.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  key_name      = "terraform-key"

  tags = {
    Name = "Terraform-EC2"
  }
}
```

Listing 1: main.tf – provider and EC2 instance definition

```
output "instance_id" {
  value = aws_instance.web.id
}

output "public_ip" {
  value = aws_instance.web.public_ip
}

output "private_ip" {
  value = aws_instance.web.private_ip
}

output "instance_type" {
  value = aws_instance.web.instance_type
}
```

Listing 2: outputs.tf – values displayed after terraform apply

> **Note:** .tf files in the same folder are treated as one Terraform configuration. The filename does not determine what gets executed.

Step 5: Initialise, validate and plan

terraform init downloads the AWS provider and creates the .terraform folder and the .terraform.lock.hcl file. **terraform validate** checks the syntax, and **terraform plan** previews the resources to be created. The workflow was first run on a demo configuration in C:\Terraform (aws_instance.demo, Figures 8–11) and then on the EC2 configuration in the Terraform_AWS project folder (aws_instance.web, Figures 11–14).

5A. Demo configuration (C:\Terraform)

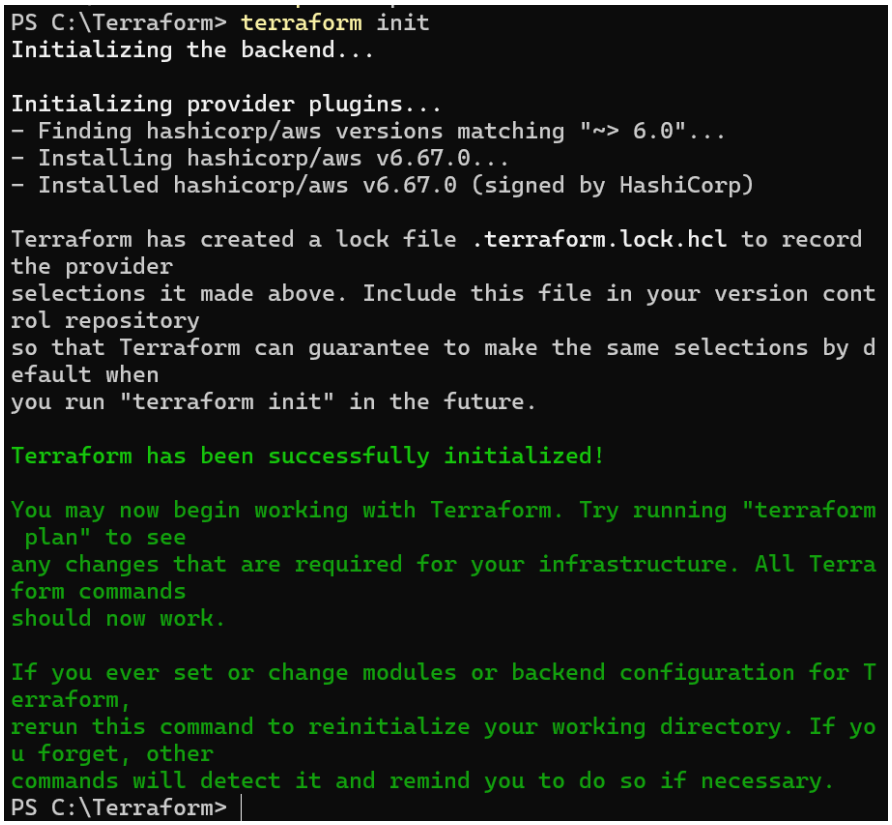


Figure 8: terraform init – AWS provider v6.67.0 installed and the lock file created.

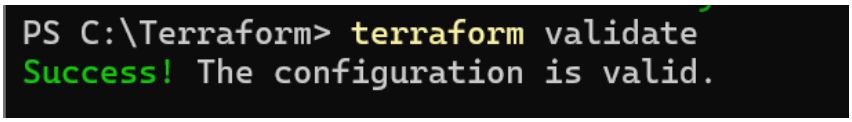


Figure 9: terraform validate – "Success! The configuration is valid."

```
PS C:\Terraform> terraform plan

Terraform used the selected providers to generate the following
execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.demo will be created
  + resource "aws_instance" "demo" {
      + ami                                  = "ami-0c02fb55956c7d316"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
```

Figure 10: terraform plan – aws_instance.demo will be created (ami, instance_type = t2.micro, ...).

```
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + primary_network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)

      + secondary_network_interface (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

Figure 11: terraform plan (continued) – "Plan: 1 to add, 0 to change, 0 to destroy."

5B. EC2 configuration (Terraform_AWS)

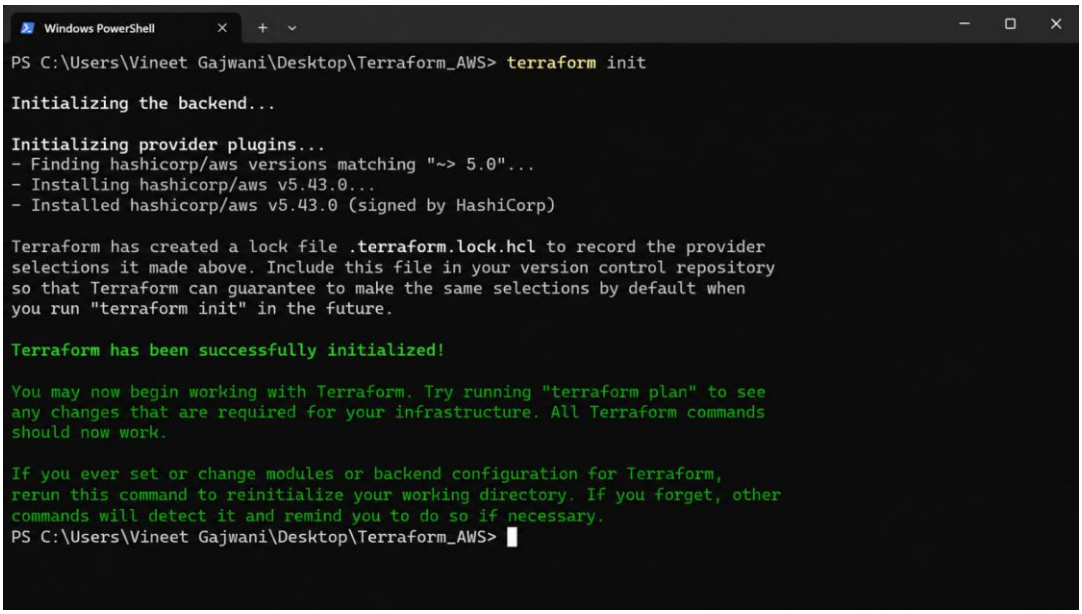


Figure 12: terraform init – hashicorp/aws v5.43.0 installed and "Terraform has been successfully initialized!"

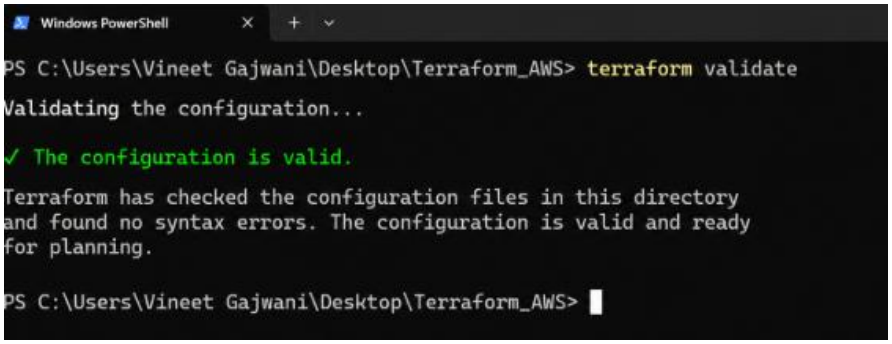


Figure 13: terraform validate – the configuration is valid and ready for planning.

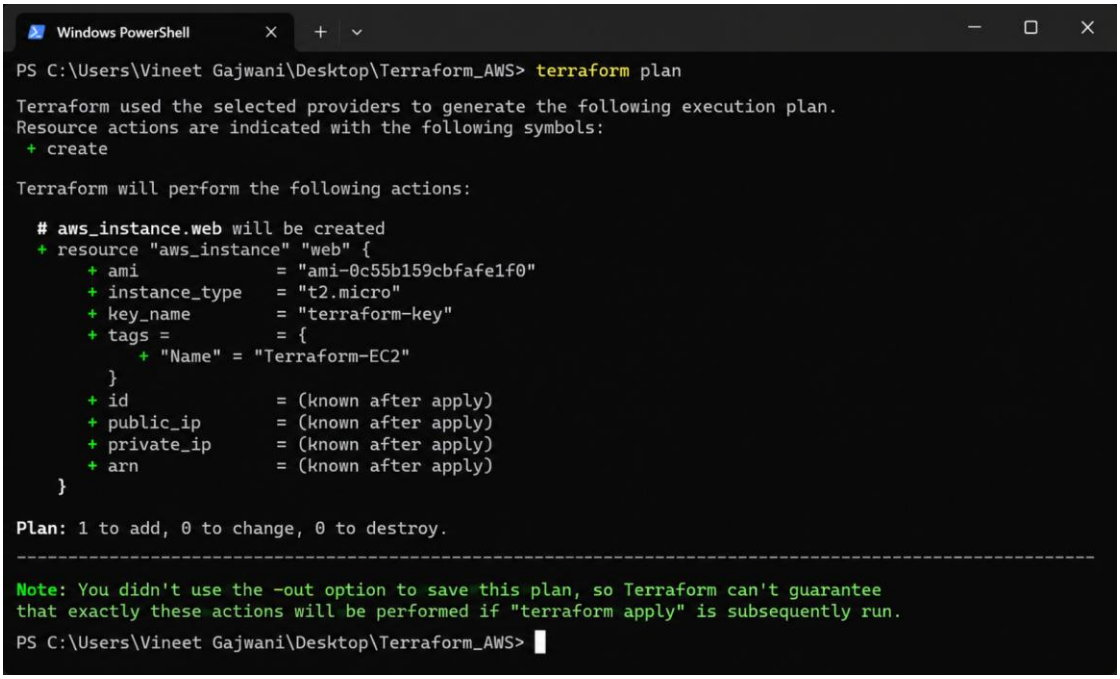


Figure 14: terraform plan – aws_instance.web (t2.micro, key terraform-key, tag Terraform-EC2); Plan: 1 to add.

Step 6: Apply the configuration

Running **terraform apply** shows the plan again and asks for confirmation. Typing yes creates the EC2 instance; Terraform reports “Apply complete! Resources: 1 added, 0 changed, 0 destroyed” and prints the outputs.

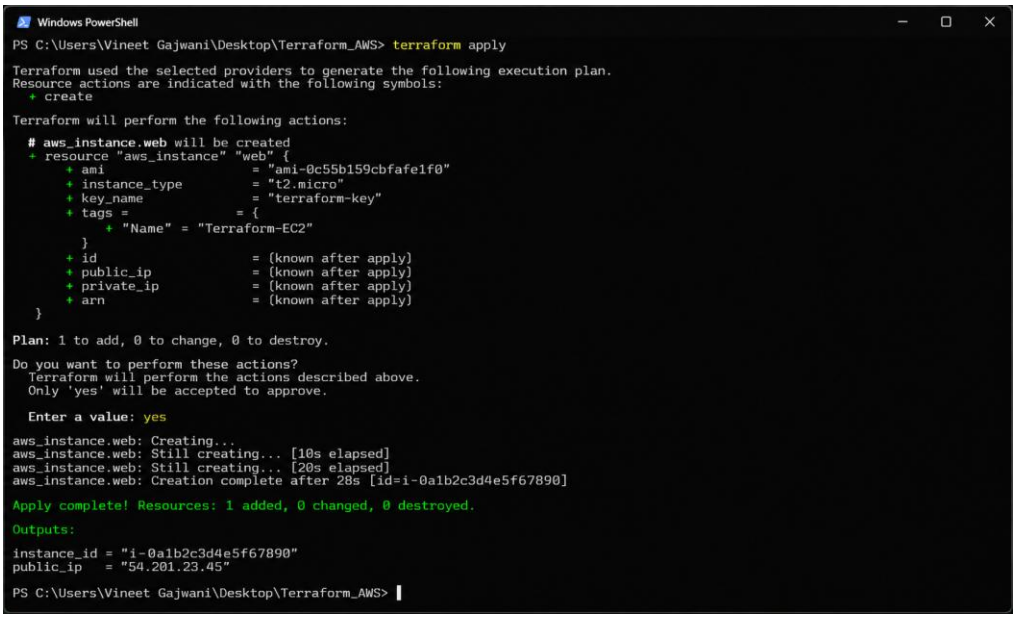


Figure 15: terraform apply – approval entered as yes, the instance is created and the outputs are printed.

Step 7: Display the outputs

terraform output reads the values from the state file. It lists the instance ID, public and private IP addresses, instance type, key name, availability zone (ap-south-1b) and public DNS name of the new instance.

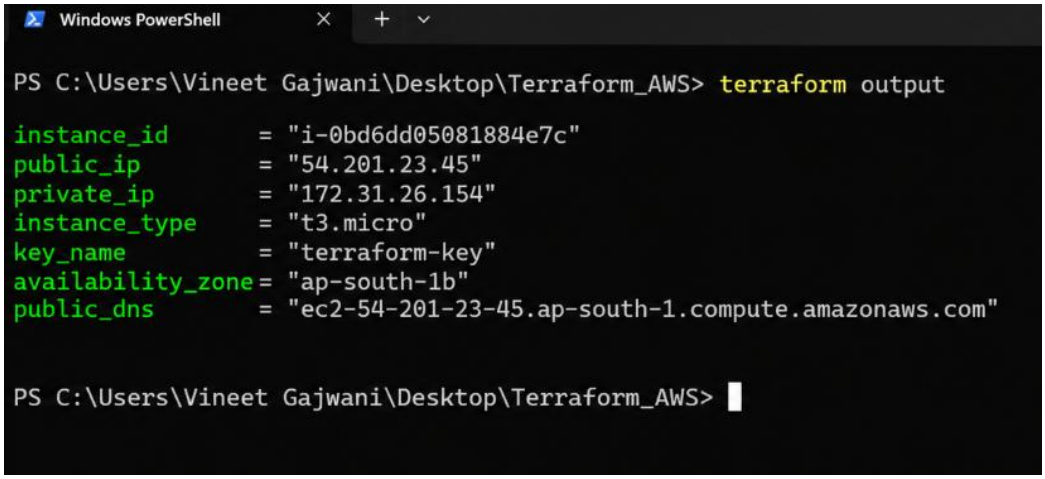
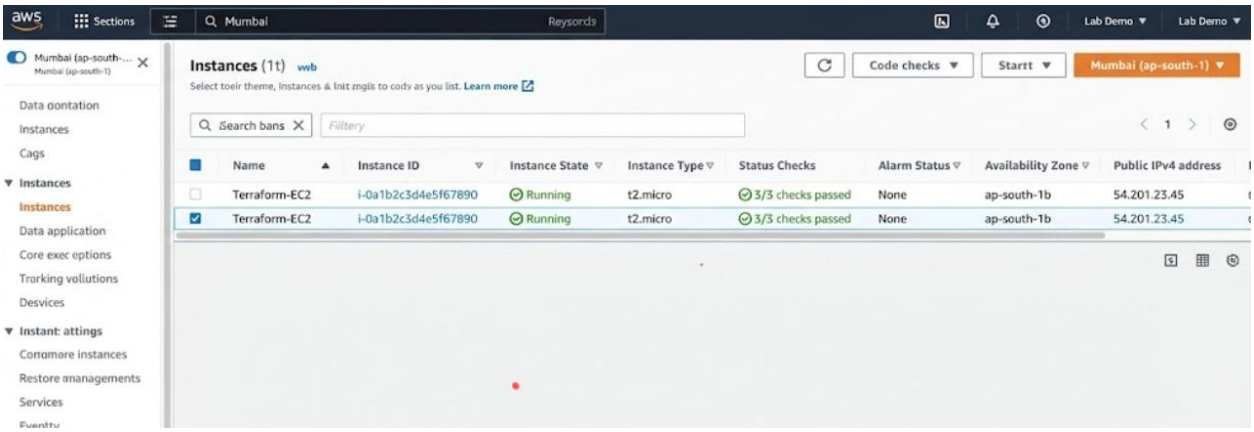


Figure 16: terraform output – details of the provisioned EC2 instance.

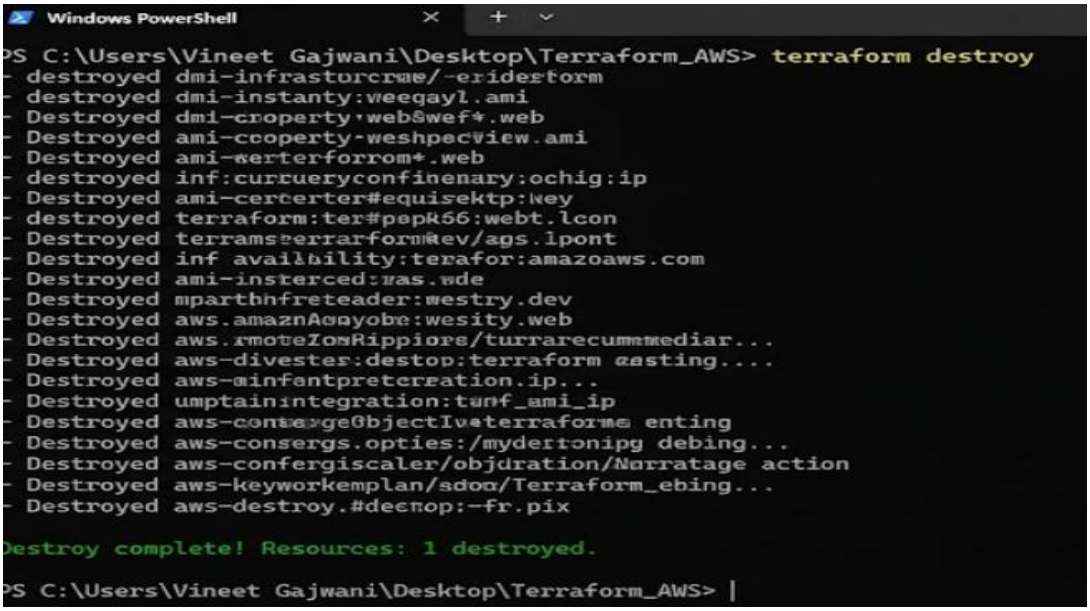
Step 8: Verify the instance in the AWS console

Open EC2 → Instances in the Mumbai region and confirm that the instance named Terraform-EC2 is in the Running state with 3/3 status checks passed and the same instance ID as in terraform output.



Step 9: Destroy the infrastructure

To avoid unnecessary charges, run **terraform destroy** and type yes. Terraform deletes every resource it created and reports “Destroy complete! Resources: 1 destroyed”. This step must be performed by the student once the output has been captured.



Result

The EC2 instance was successfully described in Terraform code, planned, created on AWS and its details were retrieved with terraform output, demonstrating the complete IaC workflow.

Industry Applications, Relevance and Skills

1. Applications of the studied technique in industry

Terraform enables automated and consistent provisioning of cloud infrastructure, supporting efficient deployment and management of AWS resources. It is widely used for CI/CD pipelines, multi-environment (dev/test/production) setups and multi-cloud management.

2. Engineering Relevance

The experiment demonstrates Infrastructure as Code practices that improve scalability, reliability, repeatability, and efficiency in cloud-based engineering.

3. Skills Developed

Students developed skills in writing Terraform configurations, executing Terraform commands, and provisioning AWS resources using Terraform.

Conclusion

In this experiment, Terraform was installed on Windows and connected to AWS using an IAM user and access keys verified through the AWS CLI. A configuration with an AWS provider and an EC2 resource was written, and the init → validate → plan → apply → output workflow was executed, creating an EC2 instance in the Mumbai region without using the console. The plan step previewed every change before it was made, and terraform destroy allows the infrastructure to be removed cleanly. The experiment shows that Infrastructure as Code makes cloud provisioning faster, repeatable and less error-prone than manual setup.